

Graded Project – Building with Claude Code (DevAnswers)

Introduction

In this capstone project, you will add two new features to **DevAnswers** — a full-stack Stack Overflow–style Q&A application for developers — by driving **Claude Code** through the agentic development workflow you learned in the **Week 11 ("Building with Claude Code")**.

The starter code is the working DevAnswers application, with a React/Redux frontend and an Express/MongoDB backend, including the Gemini-powered "Improve Question" and "Summarize Answers" features. Your job is to extend it with two classic, genuinely useful features:

1. **Bookmarks** — let users save questions of interest and revisit them later from their profile.
2. **Edit your posts** — let users edit the questions and answers they authored.

This project will help you practice:

- Applying a real, end-to-end **agentic development workflow** with Claude Code: *brainstorm* → *spec* → *plan* → *implement* → *review* → *test & verify*.
- **Spec-driven development** — generating a precise, testable spec for a feature, then reviewing and tightening it before any code is written.
- Building and committing a reusable **.claude/ workflow toolkit** (project memory + custom slash commands) so the agent stays on-pattern.
- Extending an existing, layered codebase **without breaking its conventions or its tests**.

Setup and Execution

1. Download and extract the project. It contains two folders: **devanswers-frontend** and **devanswers-backend**. Open the **entire project folder** in your VS Code so Claude Code can see both packages.
2. Start the backend:
 - `cd devanswers-backend`
 - Create a `.env` file from `.env.example`. A valid **GEMINI_API_KEY** is only needed if you want the existing AI features (Improve Question / Summarize Answers) to keep working — it is **not** required for the two features in this project.
 - `npm install`
 - `npm run populate` — seed the database with sample questions, answers, and users.
 - `npm run dev`
3. Start the frontend (in a new terminal).

- `cd devanswers-frontend`
- `npm install`
- `npm run dev`

4. Open the app in your browser (e.g. `http://localhost:5173`). Confirm the app is fully functional — you can register/log in, browse questions, open a question, answer, and vote — **before** you start building.

The Workflow You Should Use (Recommended)

Run **each feature** through the Week 11 pipeline. Every stage produces a concrete artifact you review before moving on — the agent does the mechanical work; you own the judgment.

0. **Toolkit** — set up your reusable `.claude/` toolkit: project memory plus the custom slash commands you'll drive the pipeline with (e.g. `/spec`, `/plan-feature`). You can reuse the same toolkit that was used in the mentored learning session.
1. **Brainstorm** — in Plan Mode, explore the codebase and map each feature against the existing architecture before writing any code.
2. **Spec** — generate a precise, testable spec for each feature, then review and tighten it (the highest-leverage step).
3. **Plan** — turn each spec into an ordered, bottom-up implementation plan.
4. **Implement** — build to the plan, preferably tests-first.
5. **Review** — run an automated review on the diff and address what matters.
6. **Test & verify** — run the full test suite and manually verify the feature in the running app.

Stages 0, 2, and 3 produce the artifacts you must submit: the `.claude/` toolkit (the custom commands and your `settings.json`) and, in `specs/`, a spec document plus an implementation plan for **each** feature. Treat these as real deliverables, not throwaway scaffolding — a reviewer should be able to read your spec and plan and understand exactly what you built and why.

Optionally, practise the full team workflow on top of this: a git repo, a feature branch (and `git worktree`) per feature, and a real GitHub pull request per feature via the `gh` CLI. This is **not required for submission**, but it is excellent practice.

The single biggest lever on the agent's output quality is the **spec**. A vague prompt produces vague code; a precise, reviewed spec produces correct, reviewable code. Spend your time there.

Features to Implement

The requirements below describe the **expected behaviour** — the bar your final code must meet. Turning these into a detailed, testable spec (acceptance criteria, edge cases, API contract, file list) is your job in Stage 2.

Feature 1 — Bookmark Questions for Later

Let a logged-in user save (bookmark) questions they want to return to, and revisit them from their profile.

Expected behaviour

- Each question shows a **bookmark control** (e.g. a bookmark icon) wherever questions are displayed — both in the question list/feed and on the individual question page. Its appearance makes clear whether the question is currently saved (for example, an outline vs. a filled icon).
- A logged-in user can **save** a question and later **unsave** it. Toggling updates the icon immediately, without a full page reload.
- Bookmarking requires authentication — an **anonymous visitor cannot save** questions, consistent with the rest of the app (posting and voting also require auth).
- Saved questions are **per-user**: one user's saved set never affects another user's.
- The user can see all their saved questions in one place — a **"Saved Questions" section on their profile page** — rendered with the **existing question-list UI**. From there they can open a saved question or unsave it.
- When the user has saved nothing, the profile shows a friendly **empty state** (e.g. *"No saved questions yet"*).
- On login / app load (with a valid session), the app knows which questions are already saved, so their bookmark icons render in the saved state right away.

Feature 2 — Edit Your Own Questions and Answers

Let an author edit the content of questions and answers they have posted.

Expected behaviour

- Editing is available **only on the individual question page** — the page that displays a single question together with its answers. The edit affordances must **not** appear on the home feed or any other list view.
- On a **question** they authored, the author sees an **edit affordance** — for example, a small **pencil icon** rather than a full "Edit" button. Activating it lets them change the **title, description, and tags**. Saving persists the change and the updated content is shown immediately, without a full page reload.
- On an **answer** they authored (shown on that same question page), the author sees an **edit affordance** (e.g. a pencil icon) next to their answer. Activating it lets them change the **answer text**. Saving persists and updates the answer in place.
- **Only the author** may edit their own post. A user must **not** see the edit affordance on someone else's question or answer, and the server must reject an edit attempted by anyone who is not the author.
- After a post has been edited, it shows an **"edited" indicator**. A post that has never been edited does **not** show this indicator.
- The edit form is **pre-filled** with the current content. The user can **cancel** to discard changes. Saving **blank/invalid** content (e.g. an empty title or empty body) is prevented.

Implementation Guidelines

- **Explore before you build.** Start each feature in Plan Mode and map it against the existing code. You will find the codebase already follows strict patterns — and that **some backend capability may already exist**. Build only what is genuinely missing; do not duplicate or re-invent what is already there.
- **Follow the existing architecture.** Backend changes must respect the **routes → controllers → services → models** layering, the **{ success, message, data }** response envelope, and the shared error-creation helper. Frontend changes must use the existing endpoint-config → service → Redux slice → component pattern. The **CLAUDE.md** you generate in Stage 0 should capture these so the agent stays on-pattern.
- **Don't break what works.** The existing question/answer/auth/vote flows and the AI features must keep functioning, and the existing backend and frontend test suites must still pass. Writing tests for your new code (tests-first) is strongly recommended as part of the workflow, though new tests are not themselves a graded deliverable.

Out of scope

- **Deleting** questions or answers (only editing is in scope for this project).
- Bookmarking **answers** (only questions can be saved); folders, tags, notes, sharing, or exporting of saved questions.

What "Done" Looks Like

Your submission is complete when, in the running app:

1. A logged-in user can save and unsave questions (the bookmark icon reflects the current state), sees all their saved questions in a section on their profile (with a friendly empty state), and their saves are per-user and persist across page loads.
2. On the individual question page, an author can edit their own questions (title/description/tags) and their own answers (text) via an edit affordance (e.g. a pencil icon), the changes appear immediately, edited posts show an accurate "edited" indicator, and both the UI and the server correctly restrict editing to the author.
3. All previously existing functionality and tests still work.